



CRUD en Laravel 12

Paso a paso basado en el ejercicio de rutas y vistas

Documento guía para clase

Objetivo: transformar el ejercicio original de menú de rutas y vistas en un CRUD real con base de datos, usando Laravel 12, migraciones, modelo, controlador resource y vistas Blade.

1. Punto de partida del ejercicio original

- El PDF base trabaja un mini-sitio con estas rutas: /, /blog, /blog/create, /blog/{post} y /blog/category/{category?}.
- Primero se simulan los posts en un arreglo dentro del controlador MenuController y luego se muestran en vistas Blade.
- En este nuevo desarrollo ya no vamos a usar datos simulados: construiremos un CRUD completo conectado a MySQL.

2. Resultado que vamos a construir

- Un listado de posts.
- Un formulario para crear registros.
- Una vista para mostrar el detalle de cada post.
- Un formulario para editar.
- La opción para eliminar.
- Persistencia real en base de datos con la tabla posts.

3. Crear el proyecto

- Si el proyecto aún no existe, créalo con Composer y entra a la carpeta del proyecto.



```
composer create-project laravel/laravel crud-blog
cd crud-blog
```

Luego inicia el servidor local:

```
php artisan serve
```

4. Configurar la base de datos en .env

Edita el archivo .env y ajusta los datos según tu servidor MySQL.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=crud_blog
DB_USERNAME=root
DB_PASSWORD=
```

5. Crear modelo, migración y controlador

Usaremos un recurso llamado Post. Con un solo comando Laravel generará el modelo, la migración y el controlador resource.

```
php artisan make:model Post -mcr
```

6. Configurar la migración de posts

Abre el archivo de migración generado en database/migrations y reemplaza su contenido por el siguiente:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```



```
return new class extends Migration
{
    public function up(): void
    {
        Schema::create('posts', function (Blueprint $table) {
            $table->id();
            $table->string('title');
            $table->string('slug')->unique();
            $table->text('content');
            $table->string('category')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('posts');
    }
};
```

Después ejecuta la migración:

```
php artisan migrate
```

7. Configurar el modelo Post

Edita app/Models/Post.php para permitir asignación masiva de los campos del formulario.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    protected $fillable = [
        'title',
        'slug',
        'content',
    ];
}
```



```
        'category',  
    ];  
}
```

8. Definir las rutas web

En el ejercicio original las rutas se definían manualmente y había que cuidar el orden entre rutas estáticas y dinámicas. En el CRUD usaremos `Route::resource()`.

```
<?php  
  
use Illuminate\Support\Facades\Route;  
use App\Http\Controllers\PostController;  
  
Route::get('/', function () {  
    return redirect()->route('posts.index');  
});  
  
Route::resource('posts', PostController::class);
```

Rutas que Laravel crea automáticamente

Método	Ruta	Acción
GET	/posts	index
GET	/posts/create	create
POST	/posts	store
GET	/posts/{post}	show
GET	/posts/{post}/edit	edit
PUT/PATCH	/posts/{post}	update
DELETE	/posts/{post}	destroy

9. Programar el controlador PostController

Edita `app/Http/Controllers/PostController.php` con la lógica completa del CRUD.



```
<?php

namespace App\Http\Controllers;

use App\Models\Post;
use Illuminate\Http\Request;

class PostController extends Controller
{
    public function index()
    {
        $posts = Post::latest()->get();
        return view('posts.index', compact('posts'));
    }

    public function create()
    {
        return view('posts.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'title' => 'required|min:3|max:255',
            'slug' => 'required|min:3|max:255|unique:posts,slug',
            'content' => 'required|min:10',
            'category' => 'nullable|max:100',
        ]);

        Post::create($request->all());

        return redirect()->route('posts.index')
            ->with('success', 'Post creado correctamente.');
```

```
    }

    public function show(Post $post)
    {
        return view('posts.show', compact('post'));
    }

    public function edit(Post $post)
    {
        return view('posts.edit', compact('post'));
    }

    public function update(Request $request, Post $post)
    {
        $request->validate([
```



```
        'title' => 'required|min:3|max:255',
        'slug' => 'required|min:3|max:255|unique:posts,slug,' .
$post->id,
        'content' => 'required|min:10',
        'category' => 'nullable|max:100',
    ]);

    $post->update($request->all());

    return redirect()->route('posts.index')
        ->with('success', 'Post actualizado correctamente.');
```

```
public function destroy(Post $post)
{
    $post->delete();

    return redirect()->route('posts.index')
        ->with('success', 'Post eliminado correctamente.');
```

```
}
```

10. Crear las vistas Blade

Crea la carpeta `resources/views/posts` y agrega estos archivos: `layout.blade.php`, `index.blade.php`, `create.blade.php`, `show.blade.php` y `edit.blade.php`.

10.1 layout.blade.php

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>@yield('title', 'CRUD Posts')</title>
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <style>
        body { font-family: Arial, sans-serif; margin: 30px; background:
#f8f9fa; }
        nav a { margin-right: 15px; text-decoration: none; color:
#0d6efd; font-weight: bold; }
        .container { background: #fff; padding: 20px; border-radius:
10px; margin-top: 20px; box-shadow: 0 2px 8px rgba(0,0,0,.08); }
        .success { background: #d1e7dd; color: #0f5132; padding: 10px;
border-radius: 6px; margin-bottom: 15px; }
```



```
.error { background: #f8d7da; color: #842029; padding: 10px;
border-radius: 6px; margin-bottom: 15px; }
    input, textarea { width: 100%; padding: 10px; margin-top: 5px;
margin-bottom: 15px; }
    button { padding: 10px 16px; border: none; background: #0d6efd;
color: white; border-radius: 6px; cursor: pointer; }
</style>
</head>
<body>
    <nav>
        <a href="{{ route('posts.index') }}">Inicio</a>
        <a href="{{ route('posts.create') }}">Crear Post</a>
    </nav>

    <div class="container">
        @if(session('success'))
            <div class="success">{{ session('success') }}</div>
        @endif

        @if($errors->any())
            <div class="error">
                <ul>
                    @foreach($errors->all() as $error)
                        <li>{{ $error }}</li>
                    @endforeach
                </ul>
            </div>
        @endif

        @yield('content')
    </div>
</body>
</html>
```

10.2 index.blade.php

```
@extends('posts.layout')
@section('title', 'Listado de Posts')
@section('content')
    <h1>Listado de Posts</h1>

    @if($posts->count())
        <table border="1" cellpadding="8">
            <thead>
                <tr>
                    <th>ID</th>
```



```
        <th>Título</th>
        <th>Slug</th>
        <th>Categoría</th>
        <th>Acciones</th>
    </tr>
</thead>
<tbody>
    @foreach($posts as $post)
        <tr>
            <td>{{ $post->id }}</td>
            <td>{{ $post->title }}</td>
            <td>{{ $post->slug }}</td>
            <td>{{ $post->category }}</td>
            <td>
                <a href="{{ route('posts.show', $post)
}}">Ver</a> |
                <a href="{{ route('posts.edit', $post)
}}">Editar</a>
            </td>
        </tr>
    @endforeach
</tbody>
</table>
@else
    <p>No hay posts registrados.</p>
@endif
@endsection
```

10.3 create.blade.php

```
@extends('posts.layout')
@section('title', 'Crear Post')
@section('content')
    <h1>Crear Post</h1>

    <form action="{{ route('posts.store') }}" method="POST">
        @csrf
        <label>Título</label>
        <input type="text" name="title" value="{{ old('title') }}">

        <label>Slug</label>
        <input type="text" name="slug" value="{{ old('slug') }}">

        <label>Categoría</label>
        <input type="text" name="category" value="{{ old('category') }}">
```



```
<label>Contenido</label>
<textarea name="content" rows="6">{{ old('content') }}</textarea>

<button type="submit">Guardar</button>
</form>
@endsection
```

10.4 show.blade.php

```
@extends('posts.layout')
@section('title', 'Detalle del Post')
@section('content')
    <h1>{{ $post->title }}</h1>
    <p><strong>Slug:</strong> {{ $post->slug }}</p>
    <p><strong>Categoría:</strong> {{ $post->category }}</p>
    <p><strong>Contenido:</strong></p>
    <p>{{ $post->content }}</p>
    <a href="{{ route('posts.edit', $post) }}">Editar</a>
@endsection
```

10.5 edit.blade.php

```
@extends('posts.layout')
@section('title', 'Editar Post')
@section('content')
    <h1>Editar Post</h1>

    <form action="{{ route('posts.update', $post) }}" method="POST">
        @csrf
        @method('PUT')

        <label>Titulo</label>
        <input type="text" name="title" value="{{ old('title', $post->title) }}">

        <label>Slug</label>
        <input type="text" name="slug" value="{{ old('slug', $post->slug) }}">

        <label>Categoría</label>
        <input type="text" name="category" value="{{ old('category', $post->category) }}">

        <label>Contenido</label>
```



```
<textarea name="content" rows="6">{{ old('content', $post-  
>content) }}</textarea>  
  
<button type="submit">Actualizar</button>  
</form>  
@endsection
```

11. Probar el CRUD

1. Abre en el navegador la ruta /posts.
2. Haz clic en Crear Post y registra un nuevo post.
3. Verifica que aparezca en el listado.
4. Ingresa al detalle del post.
5. Edita el registro y guarda los cambios.
6. Finalmente elimina el post para comprobar el flujo completo.

12. Explicación pedagógica para clase

- **Migración:** crea la estructura física de la tabla en la base de datos.
- **Modelo:** representa la tabla posts y permite interactuar con Eloquent.
- **Controlador:** concentra la lógica del CRUD.
- **Rutas:** conectan las URLs con cada método del controlador.
- **Vistas Blade:** muestran la interfaz al usuario.
- **Validación:** evita guardar datos incompletos o duplicados, como el slug repetido.